Resumen-Completo-Examen-Frontend...

PepaG

Introducción a la Ingeniería del Software y los Sistemas de Información II

2º Grado en Ingeniería Informática - Ingeniería del Software

Escuela Técnica Superior de Ingeniería Informática
Universidad de Sevilla

Accede al documento original

Reservados todos los derechos.
No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Guia de examen IISSI2

Flujo de pensamiento para React Native, endpoints, formularios y CRUD

Esta guia resume el razonamiento que debes seguir en el examen. No esta pensada para memorizar bloques de codigo, sino para saber que pieza toca modificar, por que se modifica y que datos deben moverse entre pantallas, estados, formularios y backend.

1. Mapa mental general

? Primero identifica la entidad: restaurante, producto, categoria o usuario.

? Despues identifica la operacion: listar, ver detalle, crear, editar o borrar.

? Traduce la operacion a HTTP: GET lista/detalle, POST crea, PUT edita, DELETE borra.

? Localiza el fichero de endpoints de esa entidad: RestaurantEndpoints.js o ProductEndpoints.js.

? Localiza la pantalla donde ocurre la accion: listado, detalle, create o edit.

? Decide que estado necesitas: lista, objeto detalle, categorias, errores, item a borrar o valores iniciales.

? Decide cuando cargar datos: normalmente en useEffect.

? Decide que se renderiza: FlatList, ImageCard, formulario, DropDownPicker, Switch, ImagePicker o DeleteModal.

? Decide que ocurre al pulsar: navegar, enviar formulario, abrir modal, confirmar borrado o refrescar datos.

2. Tabla rapida de operaciones

Quiero...	HTTP	Endpoint mental	Pantalla tipica	Idea clave
Listar restaurantes	GET	users/myrestaurants	RestaurantsScreen	Array en estado + FlatList.
Ver detalle restaurante	GET	restaurants/{id}	RestaurantDetailScreen	El id viene por route.params.id.
Crear restaurante	POST	restaurants	CreateRestaurantScreen	Formik recoge values y create(values).
Crear producto	POST	products	CreateProductScreen	Aqui si se manda restaurantId.
Editar restaurante	PUT	restaurants/{id}	EditRestaurantScreen	Cargar detalle antes de rellenar initialValues.
Editar producto	PUT	products/{id}	EditProductScreen	No mandar restaurantId.
Borrar restaurante	DELETE	restaurants/{id}	RestaurantsScreen	DeleteModal + remove + refrescar.
Borrar producto	DELETE	products/{id}	RestaurantDetailScreen	Igual, pero refrescando detalle.

3. Listar entidades

Cuando piden listar algo, piensa siempre: endpoint -> estado -> useEffect -> FlatList.

? En el endpoint debe existir una funcion que haga GET y devuelva la lista.

? En la pantalla creo un estado con array vacio para guardar la lista.

? En useEffect llamo a una funcion async que hace await al endpoint.

? Si la peticion va bien, hago setEstado con los datos recibidos.

? Si puede fallar, uso try/catch y showMessage.

? En el return uso FlatList con data, renderItem y keyExtractor.

? Si una fila debe navegar, el onPress pasa el id de ese item a la siguiente pantalla.

4. Ver detalles

Una pantalla de detalle necesita saber que entidad concreta mostrar. Esa informacion suele venir de la pantalla anterior mediante navigation.navigate.

? En la pantalla anterior navego pasando { id: item.id }.

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

? En la pantalla de detalle leo route.params.id.

? En endpoints debe existir getDetail(id).

? Creo un estado para el objeto completo, normalmente {}.

? En useEffect pido el detalle y hago setEntidad.

? Luego renderizo sus campos y, si contiene una lista hija, la pinto con FlatList.

5. Navegacion y route.params.id

El nombre id es generico. Para no equivocarte, pregunta siempre desde que item estas navegando.

? Si estoy en una lista de restaurantes, item.id es id de restaurante.

? Si estoy en una lista de productos, item.id es id de producto.

? Si entro a RestaurantDetailScreen desde RestaurantsScreen, route.params.id es restaurante.

? Si entro a EditProductScreen desde RestaurantDetailScreen, route.params.id es producto.

? Cuando tengas dudas, mira el navigation.navigate que abre esa pantalla.

6. Formularios con Formik

Formik es el cerebro del formulario: guarda values, valida y ejecuta onSubmit.

? initialValues define los nombres de los campos y sus valores iniciales.

? Los nombres deben coincidir con lo que espera el backend.

? validationSchema define reglas con yup.

? onSubmit apunta a la funcion que se ejecuta al guardar.

? handleSubmit debe ir en el Pressable de Save.

? values contiene los valores actuales.

? setFieldValue se usa para DropDownPicker, Switch e ImagePicker.

? backendErrors se usa para mostrar errores que vienen del servidor.

7. Crear entidades: POST

Crear es enviar datos nuevos al backend. La pantalla no necesita cargar una entidad previa, solo categorias si hay dropdowns.

? En endpoints creo create(data) con post.

? En la pantalla defino initialValues.

? Si hay categorias, las cargo en useEffect.

? El formulario recoge values.

? Al pulsar Save se ejecuta create(values).

? Si sale bien, muestro mensaje y navego a la pantalla que debe refrescarse.

? Si sale mal, guardo error.errors en backendErrors.

? Caso especial: al crear producto si se incluye restaurantId.

8. Editar entidades: PUT

Editar combina dos pasos: primero GET para rellenar el formulario, luego PUT para enviar cambios.

? Navego a la pantalla de edicion pasando el id de la entidad.

? Creo estado para la entidad original y para initialValues.

? En useEffect hago getDetail(route.params.id).

? Si hay imagenes, preparo con prepareEntityImages.

? Uso buildInitialValues para crear valores de Formik.

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

? Formik debe tener enableReinitialize.

? Al guardar, llamo a update(entidad.id, values).

? Si sale bien, navego atras y mando dirty true si quiero refrescar.

? Caso especial: al editar producto NO se manda restaurantId.

9. Borrar entidades: DELETE y DeleteModal

Borrar no debe hacerse directamente al pulsar Delete. Primero se guarda que item se quiere borrar y se pide confirmacion.

? En endpoints creo remove(id) con destroy.

? En la pantalla creo itemToBeDeleted inicializado a null.

? El boton Delete hace setItemToBeDeleted(item).

? DeleteModal se muestra si ese estado no es null.

? onCancel vuelve a poner el estado en null.

? onConfirm llama a removeEntity(itemToBeDeleted).

? removeEntity hace await remove(item.id), refresca, cierra modal y muestra mensaje.

10. DropDownPicker

Sirve para que el usuario vea nombres, pero el backend reciba ids.

? Importo DropDownPicker.

? Creo estado open y estado para opciones.

? Cargo las categorias en useEffect.

? Transformo cada categoria a { label: e.name, value: e.id }.

? value debe ser values.restaurantCategoryId o values.productCategoryId.

? onSelectItem usa setFieldValue con el nombre exacto.

? Debajo pongo ErrorMessage.

? Si pisa lo de abajo, revisa marginBottom.

11. Switch

Sirve para booleanos, por ejemplo availability.

? Importo Switch desde react-native.

? El campo booleano debe existir en initialValues.

? En yup puede aparecer como yup.boolean().

? value es values.availability.

? onValueChange actualiza Formik con setFieldValue.

? Si uso styles.switch, ese estilo debe existir.

? Pantalla blanca tipica: usar Switch sin importarlo.

12. ImagePicker

Encapsula elegir imagen, previsualizarla y guardar el resultado en el formulario.

? Importo ImagePicker y una imagen por defecto.

? El campo de imagen debe existir en initialValues.

? image recibe values.logo, values.heroImage o values.image.

? defaultImage es la imagen local de respaldo.

? onImagePicked guarda con setFieldValue.

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

? En edicion, si la imagen viene del backend, uso prepareEntityImages.

13. useEffect sin perderse

No pongas async directamente en useEffect.

? Dentro del useEffect defines una funcion async y luego la llamas.

? [] significa al montar la pantalla una vez.

? [loggedInUser] significa cuando cambie el usuario.

? [loggedInUser, route] tambien cuando vuelvas navegando con cambios.

? [route] significa cuando cambien parametros de navegacion.

? Si la pantalla depende de un id recibido, normalmente usa route.params.id.

14. Errores frecuentes y como pensarlos

Error	Como razonarlo
Pantalla blanca	Busca import mal escrito, componente no importado, variable no definida o ruta de asset incorrecta.
invalid restaurantId value	En editar producto no mandes restaurantId. En crear producto si se manda.
No se actualiza la lista al volver	Asegurate de navegar con dirty true y de que useEffect dependa de route.
Dropdown vacio	Revisa endpoint de categorias, estado de items y transformacion label/value.
Switch rompe pantalla	Revisa import de Switch y styles.switch.
Imagen no cambia	Revisa que setFieldValue use el nombre exacto del campo.
Botones pisados	Revisa position absolute, margenes y contenedores.
No entiendo route	Vuelve al navigate que abre la pantalla: ahi esta el significado del id.
404 o endpoint raro	En frontend se usa products/${id}; los dos puntos :id son de definicion de ruta en backend.

15. Checklist antes de entregar

? He identificado entidad y operacion.

? El endpoint existe, importa el helper correcto y esta exportado.

? La pantalla importa desde ProductEndpoints o RestaurantEndpoints segun corresponda.

? Los nombres de campos coinciden con el backend.

? Si uso route.params.id, se de que entidad es ese id.

? Si uso Formik, initialValues, validationSchema y onSubmit tienen sentido.

? Si uso DropDownPicker, cargo categorias y uso label/value.

? Si uso Switch, esta importado y el campo es booleano.

? Si uso ImagePicker, setFieldValue usa el nombre exacto.

? Si edito producto, no envio restaurantId.

? Si borro algo, uso DeleteModal y refresco despues.

? Si vuelvo a una lista tras crear/editar/borrar, la pantalla se refresca.

? He mirado consola si hay pantalla blanca.

16. Estrategia de examen

Cuando leas un enunciado, no empieces copiando codigo. Traducelo primero a una historia tecnica: el usuario pulsa aqui, necesito este id, llamo a este endpoint, guardo esto en estado, pinto esto, y si sale bien navego alli. Esa historia te dice que ficheros tocar.

? Si piden una pantalla nueva, revisa si hay que registrarla en el Stack.

? Si piden obtener datos, empieza por endpoints y useEffect.

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

? Si piden crear o editar, piensa en Formik y en nombres exactos de campos.

? Si piden borrar, piensa en remove + DeleteModal + refrescar.

? Si piden categorias, piensa en DropDownPicker y transformacion label/value.

? Si piden booleanos, piensa en Switch.

? Si algo sale blanco, abre consola y busca la primera linea real del error.

Idea final: endpoint trae o modifica datos; estado los guarda; Formik controla formularios; FlatList pinta listas; navigation mueve ids entre pantallas. Si entiendes ese circuito, puedes reconstruir casi cualquier ejercicio.

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.